

PocketSmart AI: Smart Budget & Recommendation Assistant

Project Description:

PocketSmart AI is a Generative AI-powered budget and recommendation assistant designed to help users plan everyday lifestyle needs within a defined budget. The system combines budget information, user preferences, category-specific inputs, and optional contextual information to generate personalized recommendations.

The application focuses on three major planning domains: Home Interior Planning, Party Budget Planning, and Jewelry Recommendations. Gemini is used as the AI recommendation layer to interpret the user's requirements and generate structured, context-aware suggestions.

The application is implemented with FastAPI as the backend framework, Jinja2/HTML templates for the web interface, CSS and JavaScript for the frontend experience, and SQLite for persistent application data. Authentication, sessions, recommendation history, modular routes, and reusable services are included to provide a complete user workflow.

PocketSmart AI simplifies budget planning by connecting a user's financial limit with practical recommendations, reducing the need to manually search across multiple categories.

Scenarios

Scenario 1: Home Interior Planning with Smart Budget Allocation

A user selects the Home Budget Planner and enters a total budget together with room and furnishing requirements such as living room, bedroom, kitchen, lighting, ceiling fans, furniture, and decor. PocketSmart AI processes the inputs and generates budget-aware suggestions for the requested categories, balancing functionality, style, and affordability.

Scenario 2: AI-Based Party Budget Planning

A user selects the Party Planner and provides the total budget, guest count, event type, venue requirements, and other preferences. The system organizes recommendations for food, decoration, entertainment, venue, and related services according to the available budget.

Scenario 3: Jewelry Recommendations for Occasions

A user selects the Jewelry Budget Planner and provides a budget, occasion, and style preferences. The module can also support optional contextual image information where available. Gemini uses the provided context to generate jewelry recommendations aligned with the occasion, style, and budget.

Technical Architecture:

Description: PocketSmart AI uses a modular web application architecture. The frontend is implemented using HTML/Jinja2 templates, CSS, and JavaScript. FastAPI acts as the backend service layer and manages routing, validation, authentication, sessions, planner modules, and communication with the AI service. Gemini provides the generative AI recommendation capability, while SQLite stores application data.

The architecture separates authentication, database access, routes, schemas, security, middleware, and AI services so the application can be maintained and extended independently.

Core Technologies

FastAPI: Backend framework used for routing, request handling, dependency injection, and application services.

Gemini AI: Generative AI layer used to interpret planner requirements and generate personalized recommendations.

SQLite: Lightweight relational database used for persistent application data.

Jinja2 Templates: Server-rendered HTML templates used for the web interface.

HTML/CSS/JavaScript: Frontend technologies used for forms, styling, interactions, and recommendation display.

Uvicorn: ASGI server used to run the FastAPI application.

Git and GitHub: Version control and source-code collaboration.

Render: Cloud deployment platform used to publish the application.

Component-Wise Architecture

Frontend UI: Provides the landing page, authentication pages, dashboard, planner forms, recommendation views, shopping/recommendation pages, and history interface.

FastAPI Backend: Handles application routes, request processing, authentication, sessions, planner workflows, and responses.

Authentication and Security Layer: Manages registration, login, password handling, active-user checks, session information, and protected routes.

Planner Modules: Home, Party, and Jewelry planner routes collect category-specific requirements and pass them to the recommendation service.

Gemini Service Layer: Builds the AI interaction for planner requirements and returns generated recommendations.

Database Layer: Provides SQLite connectivity and application models for persistent data.

History Module: Stores and retrieves users' previous recommendation queries/results for later review.

Project Folder Structure

```
Kaviya/
├── app/
│   ├── main.py
│   ├── auth.py
│   ├── config.py
│   ├── database.py
│   ├── middleware.py
│   ├── models.py
│   ├── schemas.py
│   ├── security.py
│   ├── routes/
│   │   ├── auth.py
│   │   ├── history.py
│   │   ├── pages.py
│   │   └── planners.py
│   └── services/
│       ├── catalog.py
│       └── gemini_service.py
├── static/
│   ├── css/style.css
│   └── js/app.js
├── templates/
│   ├── base.html
│   ├── index.html
│   ├── login.html
│   ├── register.html
│   ├── dashboard.html
│   ├── history.html
│   ├── home_planner.html
│   ├── party_planner.html
│   ├── jewelry_planner.html
│   └── shopping.html
├── requirements.txt
└── .gitignore
```

Pre-requisites:

- Python 3.x and pip
- FastAPI and Uvicorn knowledge
- Basic Python programming knowledge
- HTML, CSS, JavaScript, and Jinja2 template knowledge
- SQLite/database fundamentals
- Git and GitHub for source-code management
- Gemini API access and an API key configured securely as an environment variable
- A Render account for cloud deployment
- A modern web browser for testing the application.

Project Workflow:

Milestone 1: Gemini AI Initialization

- Activity 1.1: Configure access to the Gemini API and select the Gemini model used by the application.
- Activity 1.2: Store the Gemini API credential securely using environment configuration.
- Activity 1.3: Test AI connectivity with representative planner prompts.
- Activity 1.4: Refine prompts for budget interpretation, category selection, and recommendation generation.

Milestone 2: Core Functionalities Development

- Activity 2.1: Develop the Home Interior Planner.
- Activity 2.2: Develop the Party Budget Planner.
- Activity 2.3: Develop the Jewelry Budget Planner.
- Activity 2.4: Implement the recommendation service and category-specific prompt handling.
- Activity 2.5: Add recommendation history and reusable catalog/supporting logic.

Milestone 3: Backend – FastAPI Integration

- Activity 3.1: Create the main FastAPI application and startup configuration.
- Activity 3.2: Organize authentication, history, page, and planner routes into separate modules.
- Activity 3.3: Implement schemas, models, database access, middleware, and security utilities.
- Activity 3.4: Connect planner routes to the Gemini service.
- Activity 3.5: Configure static files and Jinja2 templates.
- Activity 3.6: Run the application through Uvicorn and validate the backend workflow.

Milestone 4: Frontend Development

- Activity 4.1: Build the common base template and landing page.
- Activity 4.2: Create registration and login interfaces.
- Activity 4.3: Create the authenticated dashboard.
- Activity 4.4: Build separate forms for Home, Party, and Jewelry planners.
- Activity 4.5: Display generated recommendations in user-friendly layouts.
- Activity 4.6: Provide shopping/recommendation and history pages.

Milestone 5: Testing, GitHub and Deployment

Activity 5.1: Test authentication, planner forms, recommendation generation, and history.

Activity 5.2: Resolve missing dependencies and validate requirements.txt.

Activity 5.3: Commit and push the completed project to GitHub.

Activity 5.4: Configure the Render deployment environment.

Activity 5.5: Redeploy after dependency/configuration corrections and verify a successful live deployment.

Milestone 6: Conclusion

Complete the documentation, record the implemented modules and deployment status, and identify possible future enhancements.

Milestone 1: Gemini AI Initialization

This milestone establishes the AI foundation of PocketSmart AI. Gemini interprets natural-language planning requirements, budgets, preferences, and contextual information and returns recommendation content. The AI configuration is kept separate from application routing so the recommendation service can be maintained independently.

Activity 1.1: Configure Gemini API Access

Create or obtain the required Gemini API credential and configure it as an environment variable. The credential should not be written directly into application source code or committed to GitHub.

Create the required AI service/API access.

Generate the API credential.

Store the credential in local environment configuration.

Ensure secret files are excluded through .gitignore.

Use the configured credential from the application's configuration/service layer.

Activity 1.2: Configure the AI Service

The project contains app/services/gemini_service.py for AI recommendation logic. This keeps AI-specific processing separate from route definitions and makes the backend modular.

Accept planner input from the relevant route.

Construct category-appropriate AI instructions.

Include budget and preference information in the request.

Process the generated recommendation response.

Return the result to the planner workflow for display and history handling.

Activity 1.3: Validate Gemini Connectivity

Test the integration with representative Home, Party, and Jewelry planner inputs. Validation should confirm that the service can communicate with Gemini and that the application can handle a generated response without breaking the user workflow.

Activity 1.4: Prompt Refinement

Prompts are refined to make recommendations more relevant to the selected category and budget.

Testing focuses on whether generated results remain aligned with user inputs, category requirements, and affordability constraints.

Milestone 2: Core Functionalities Development

This milestone implements the three central PocketSmart AI planning modules. Each planner collects domain-specific requirements and connects them to the shared recommendation service.

Activity 2.1: Home Interior Planner

- Collect the user's home/interior budget.
- Collect room and furnishing requirements.
- Capture quantities and preferences where applicable.
- Send structured requirements to the recommendation service.
- Display budget-aware home interior suggestions.

Activity 2.2: Party Budget Planner

- Collect total event budget and guest count.
- Collect event type and venue-related details.
- Process food, decoration, entertainment, and related planning requirements.
- Generate structured party recommendations.
- Present recommendations in an easy-to-review format.

Activity 2.3: Jewelry Budget Planner

- Collect jewelry budget and occasion.
- Collect style preferences and contextual details.
- Support optional image/context input where implemented.
- Send requirements to Gemini.
- Display occasion- and style-aware recommendations.

Activity 2.4: Recommendation Service and Catalog

The services layer contains reusable recommendation logic. `catalog.py` supports recommendation/catalog-related application functionality, while `gemini_service.py` handles Gemini AI integration. Keeping these responsibilities outside page routes reduces duplication and makes future category expansion easier.

Activity 2.5: Recommendation History

The history module allows authenticated users to retrieve previous recommendation queries and results. This provides continuity between sessions and allows users to review earlier planning decisions.

Milestone 3: Backend – FastAPI Integration (`main.py`)

This milestone focuses on implementing the FastAPI backend as the bridge between the browser interface, authentication/database layers, planner modules, and Gemini recommendation service. The backend is organized into separate route and service modules.

Activity 3.1: Define FastAPI Routes

- Home/pages routes: Serve the application's main pages and planner interfaces.
- Authentication routes: Handle registration, login, logout, token/session-related operations, and active-user access.
- Planner routes: Handle Home, Party, and Jewelry planning workflows.
- History routes: Retrieve stored recommendation history.

Recommendation-related routes: Process recommendation details and return generated results.

Activity 3.2: Modular Architecture Setup

app/main.py – application entry point and FastAPI application configuration.

app/auth.py – authentication-related application logic.

app/config.py – application configuration.

app/database.py – database connectivity and session setup.

app/models.py – database models.

app/schemas.py – structured request/response schemas.

app/security.py – security-related utilities.

app/middleware.py – middleware configuration.

app/routes/ – modular route definitions.

app/services/ – reusable catalog and Gemini recommendation services.

Activity 3.3: Authentication and Session Control

The application provides registration, login, logout, and authenticated-user workflows. Protected planner/history functionality can use the current-user context to personalize the application. Session information is used to maintain the user's authenticated experience.

Activity 3.4: Database Integration

SQLite is used as the application's lightweight relational database. The database layer is separated from the routes so database access and model definitions remain maintainable.

Activity 3.5: Startup and Server Execution

The FastAPI application is started through an ASGI server such as Uvicorn. The main application loads configuration and registers routes, middleware, static files, templates, and services.

Typical local execution:

```
uvicorn app.main:app --reload
```

Milestone 4: Frontend Development

The frontend provides the user-facing experience for PocketSmart AI. Jinja2 templates are used with FastAPI to render pages, while CSS and JavaScript provide styling and interactive behavior.

Activity 4.1: Develop HTML/Jinja2 Frontend Interface

base.html – shared layout/template structure.

index.html – main landing/home interface.

register.html – new user registration.

login.html – existing user login.

dashboard.html – authenticated user dashboard.

home_planner.html – Home Interior planning form.

party_planner.html – Party planning form.

jewelry_planner.html – Jewelry planning form.

shopping.html – recommendation/shopping interface.

history.html – saved recommendation history.

Activity 4.2: Dynamic Recommendation Display

The frontend collects planner inputs and presents generated AI output in structured sections. JavaScript in static/js/app.js supports client-side interactions and improves the user experience when forms and recommendation results are processed.

Activity 4.3: Responsive Styling

The stylesheet in static/css/style.css provides the application's visual design and responsive presentation.

Milestone 5: Testing, Optimization and Deployment

This milestone validates the complete application workflow and prepares the project for cloud deployment. Testing includes authentication, planner functionality, AI response handling, history, static assets, dependencies, and production startup.

Activity 5.1: Local Environment Setup

Open a terminal in the project directory.

Create a Python virtual environment.

Activate the environment.

Install project dependencies from requirements.txt.

Configure required Gemini/API environment variables.

Start the FastAPI application with Uvicorn.

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
uvicorn app.main:app --reload
```

Activity 5.2: Functional Testing

Register a new user and verify successful account creation.

Log in with valid credentials and verify authenticated access.

Test the Home Planner with different budgets and room requirements.

Test the Party Planner with different event types, budgets, and guest counts.

Test the Jewelry Planner with different occasions and styles.

Verify AI-generated results are displayed correctly.

Verify recommendation history is stored/retrieved as intended.

Test invalid or incomplete input and verify appropriate handling.

Activity 5.3: Dependency and Deployment Fixes

During deployment validation, missing Python dependencies were identified and added to requirements.txt. The final dependency configuration includes required packages for the FastAPI application and authentication/email validation workflow, including itsdangerous and email-validator. After the dependency corrections were committed and pushed to GitHub, the Render service was redeployed and the deployment completed successfully.

Activity 5.4: GitHub Version Control

Initialize the local Git repository.

Connect the project to the GitHub repository.

Commit the PocketSmart AI project files.

Push the project branch to GitHub.

Merge the completed project/dependency changes into the main branch.

Use the main branch as the source for deployment.

Activity 5.5: Render Cloud Deployment

PocketSmart AI was deployed to Render as a live web application. The Render service was configured to install project dependencies and start the FastAPI application. After correcting missing dependencies and redeploying, the service reported a successful live deployment.

- Connect the GitHub repository to the Render service.

- Configure the Python environment/build process.

- Configure required environment variables such as the Gemini API credential.

- Configure the production start command for the FastAPI/Uvicorn application.

- Trigger deployment from the main branch.

- Review build and runtime logs.

- Confirm the Render service reports a successful live deployment.

Deployment Status: Render deployment completed successfully and the application was shown as Live.

Exploring the Website's Web Pages:

Home Page

The Home Page introduces PocketSmart AI and provides navigation into the application's main planning capabilities. It acts as the entry point for users to understand the service and begin their planning workflow.

Register Page

The Register Page allows a new user to create an account. Registration information is validated and stored through the application's authentication/database workflow.

Login Page

The Login Page authenticates an existing user and establishes the authenticated session needed to access personalized application features.

User Dashboard

The User Dashboard provides the logged-in user with access to planner modules and relevant application information. It serves as a central point from which the user can continue planning activities.

Home Interior Budget Planner Page

This page collects home-related requirements such as budget, room type, quantities, furniture, lighting, and decor preferences. The submitted information is used to generate budget-aware recommendations.

Home Interior Recommendations Page

The Home recommendations interface presents AI-generated suggestions based on the user's home requirements, preferences, and budget.

Party Budget Planner Page

This page collects event-related information such as budget, guest count, event type, venue details, food, decoration, and entertainment requirements.

Party Budget Recommendations Page

The Party recommendations interface presents AI-generated options organized around the event requirements and available budget.

Jewelry Budget Planner Page

This page collects jewelry budget, occasion, style preferences, and optional contextual image information where supported.

Jewelry Budget Recommendations Page

The Jewelry recommendations interface presents personalized suggestions based on occasion, style, contextual information, and budget.

Shopping / Recommendation Page

The shopping/recommendation interface provides a user-friendly view of recommendation results and associated product/service information supported by the application.

Recommendation History Page

The Recommendation History Page displays previous recommendation queries and results associated with the authenticated user, allowing earlier plans to be reviewed or reused.

Testing Summary

Testing was performed across the major application workflows. Authentication was checked through registration and login flows, while the Home, Party, and Jewelry planners were exercised using different budgets and user inputs. Recommendation generation was checked for successful response handling and presentation.

Deployment testing also included dependency validation. When Render identified missing dependencies, the requirements file was updated, the changes were committed and merged, and the service was redeployed. The final Render deployment reported a successful live status.

Future Enhancements

- Add richer real-time product and service data integrations where supported.

- Improve recommendation ranking using user feedback and historical preferences.

- Add more planning categories such as travel, education, fitness, or personal finance.

- Provide downloadable or shareable budget plans.

- Add advanced analytics for budget utilization and recommendation history.

- Improve multimodal capabilities for additional planner categories.

- Introduce stronger automated testing and monitoring for production deployments.

Conclusion:

PocketSmart AI is a Generative AI-powered smart budget and recommendation assistant that combines FastAPI, Gemini, SQLite, Jinja2 templates, HTML/CSS/JavaScript, authentication, and modular service architecture. It addresses practical planning requirements across Home Interiors, Party Planning, and Jewelry selection by interpreting user budgets and preferences and generating personalized recommendations.

The modular backend separates routes, authentication, database access, schemas, security, middleware, and AI services, making the project easier to maintain and extend. The frontend provides dedicated planner pages, authentication screens, a dashboard, recommendation views, shopping support, and recommendation history.

The project was taken through GitHub-based version control and cloud deployment on Render. After resolving deployment dependency issues, the application was successfully redeployed and reached a Live deployment state. Overall, PocketSmart AI demonstrates how Generative AI and web technologies can be combined to support practical, budget-aware planning in a single user-friendly application.